

TN SKILLS PROJECT



DEPARTMENT OF COMPUTER APPLICATIONS

**ALPHA ARTS AND SCIENCE COLLEGE
PORUR, CHENNAI**

(Reaccredited by NAAC and Affiliated to University of Madras)

COLLEGE NM ID: UNM1429

OCTOBER 2026

PROJECT TITLE:

AI-Augmented Backend Development

AI WEATHERWISE API

TEAM DETAILS:

TEAM ID: 6ab4bf896f290be8a6b413e7 / SWTID – 2026-4469

TEAM MEMBERS:

TEAM LEADER: JAYAVARSHINI B

NM ID: CAF381BC6958D42A7054C2311E6E5A71

TEAM MEMBER: KANAGAVALLI S

NM ID: 43DB4066274811FB5BD479D7B7B9C7D7

TEAM MEMBER: KEERTHANA S

NM ID: 9E0CD16F105D26789DB453C13F846CE1

TEAM MEMBER: MANISHA R

NM ID: D22BAB6B378290B80E34E0EBF98419E5

Description:

The AI WeatherWise is a robust RESTful backend platform built with Node.js and Express.js, utilizing MongoDB and Mongoose ODM for data persistence. It delivers real-time weather metrics integrated with Google Gemini AI to provide intelligent summaries and personalized recommendations based on live environmental conditions.

The platform features secure JWT authentication, bcrypt password hashing, and a comprehensive suite of endpoints for managing favorite locations and fetching weather insights. It includes a resilient fallback mode ensuring API functionality even without external API keys, alongside centralized error handling and request sanitization.

Scenario-Based Case Study

Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather conditions and receive AI-driven activity suggestions based on current forecasts.

Problem

- Difficulty tracking and managing multiple favorite weather locations efficiently.**
- Lack of personalized recommendations for outdoor activities or clothing.**
- Need for natural language weather summaries rather than just raw data metrics.**
- Absence of a secure system to store and sync preferences across devices.**

Solution:

The AI WeatherWise provides a centralized service that handles weather data retrieval and AI orchestrations. It manages a secure User and Location database, allowing for customized insights and automated recommendation workflows through Google Gemini.

Usage

- **User Usage:** Manages favorite cities (CRUD), fetches real-time weather, and requests AI insights for recommendations and summaries.
- **Developer/Admin Usage:** Monitors API health, manages global user registrations, and verifies weather/AI service integrations.
- **Weather Application User:** Interfaces with the public weather endpoints to consume current metrics without requiring an account.

Outcome

By leveraging AI and centralized data management, users spend less time deciphering raw metrics. The system ensures data integrity via Mongoose schemas and provides high-value, context-aware weather advice through Gemini integrations.

1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Gemini AI SDK:** Integration using `@google/generative-ai` for natural language insights.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

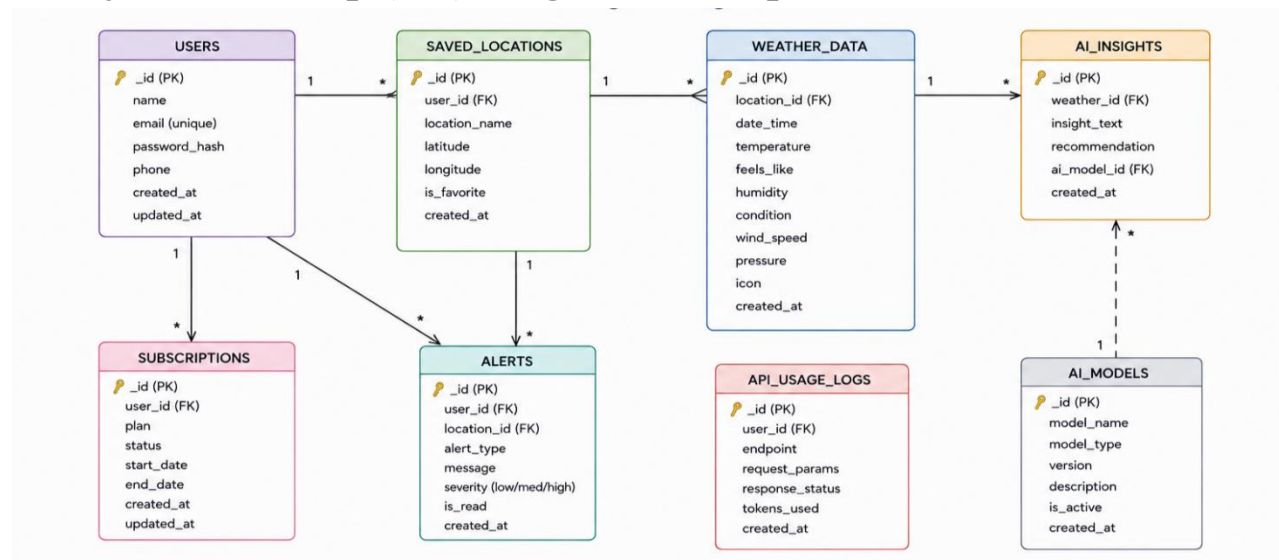
Epic 1: PROJECT ARCHITECTURE

The AI WeatherWise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages User and Location entities.

Components Definition

- **Interface:** API tools like Postman query distinct route paths to extract JSON entities.
- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.
- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.
- **AI Service Integration:** Leverages @google/generative-ai to generate weather summaries and personalized recommendations based on live temperature and conditions.

Entity-Relationship (ER) Diagram Description



The data architecture manages User profiles and their associated favorite Locations through logical reference.

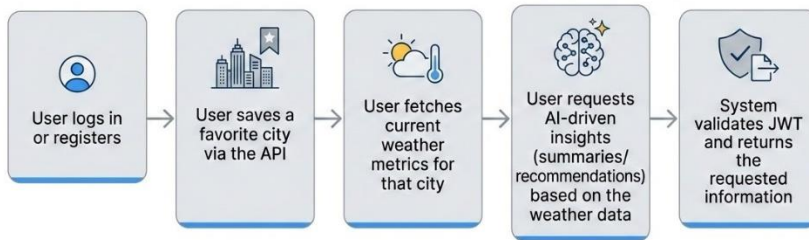
Key Features

- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths.
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability.
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.

User Flow:

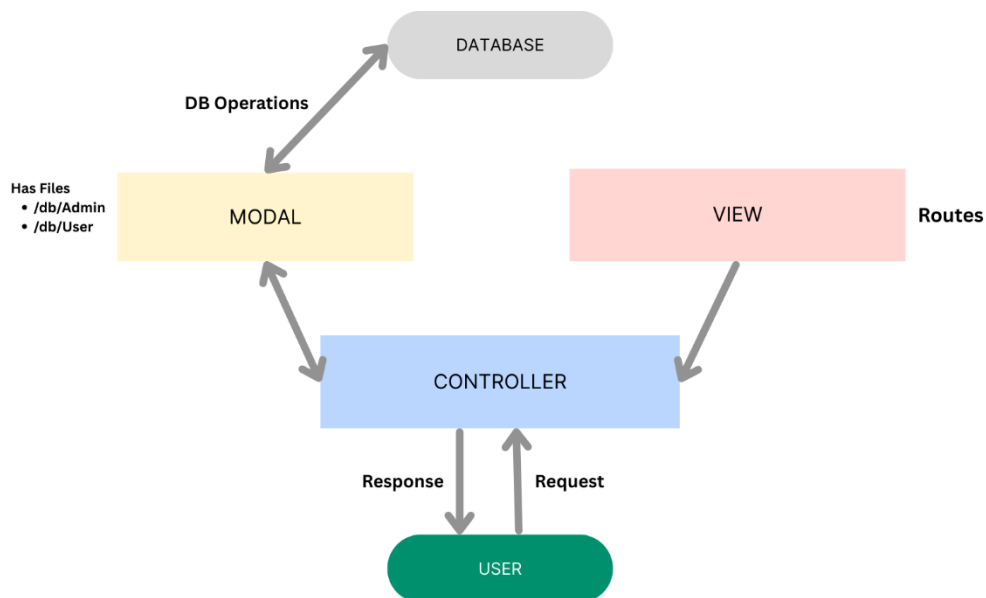
1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.
5. System validates JWT and returns the requested information.

AI WeatherWise API User Workflow



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

Epic 2: Project Setup and Configuration

Creating the Project Folder

- **Create a new folder named AI Weatherwise**
- **Open the project folder in Visual Studio Code.**
- **Initialize the project using Node.js and install the required dependencies.**

Create the Following Files

- **index.js**
- **.env**
- **package.json**
- **.gitignore**

Environment Configuration

Create a .env file in the project root and configure the required environment variables.

PORT=5000

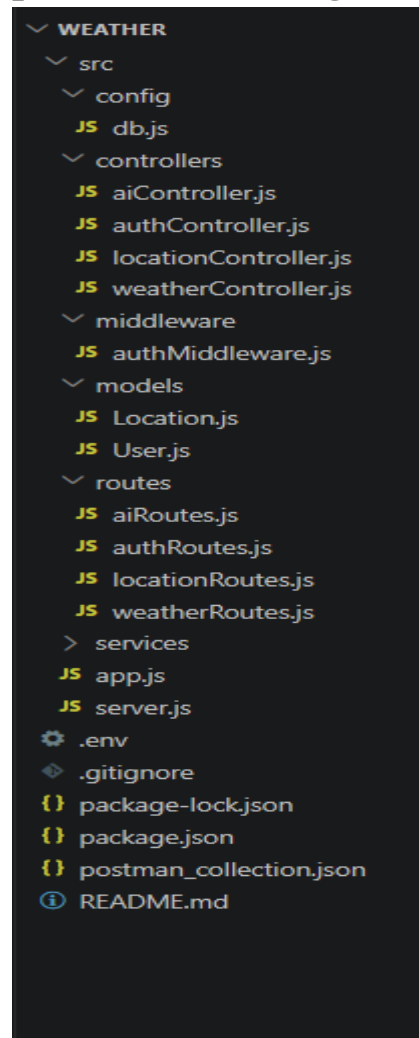
MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key

Epic 3: Backend Development

The AI weatherwise backend is developed using Node.js, Express.js, and MongoDB, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.



Backend Structure:

Core System Script Implementations

1. Application Entry Point (index.js)

The index.js file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Epic4: Database Configuration

API Testing Demo :

1. Configure MongoDB:

- Install Mongoose.

In your Node.js project directory, run:

npm install mongoose

- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below **Mongoose Schema Definitions**

Epic 5: Project Executions:

The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

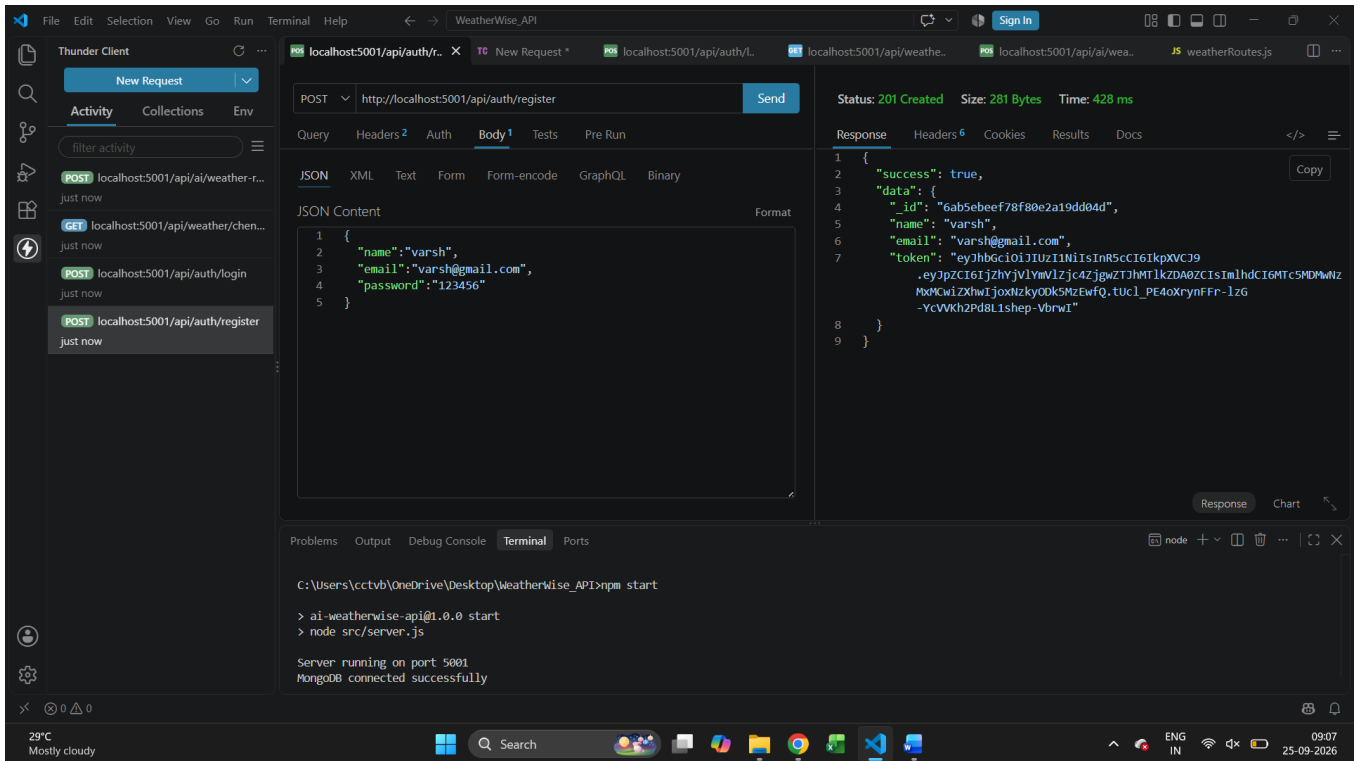
```
npm install
```

Start the development server:

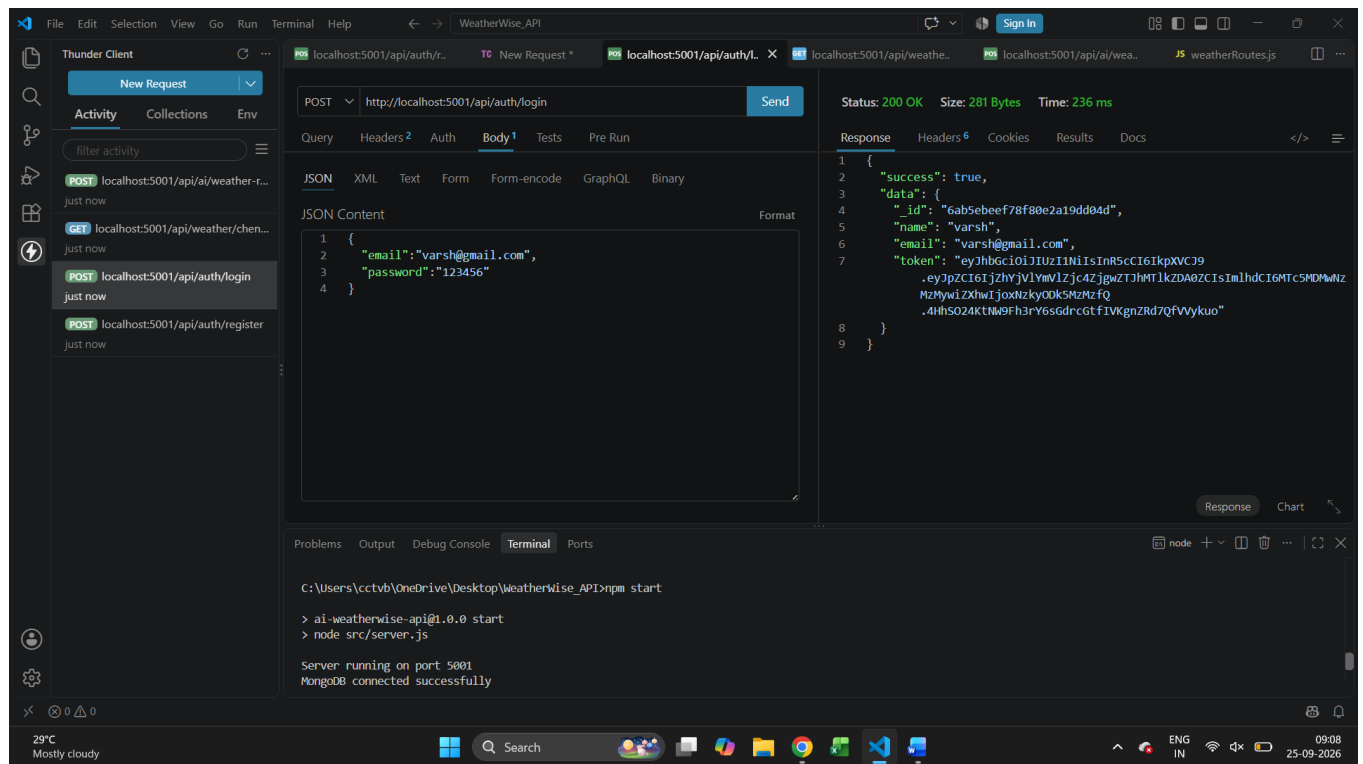
```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

REGISTER API:



LOGIN API:



WEATHER API :

The screenshot displays the Thunder Client interface with a REST client request and its response.

Request Details:

- Method: GET
- URL: http://localhost:5001/api/weather/bangalore
- Body: JSON Content (1 {})

Response Details:

- Status: 200 OK
- Size: 124 Bytes
- Time: 396 ms
- Response Body (JSON):

```
{  "success": true,  "data": {    "city": "bangalore",    "temperature": 34,    "humidity": 79,    "windSpeed": 12,    "condition": "windy",    "isMock": true  }}
```

Terminal Output:

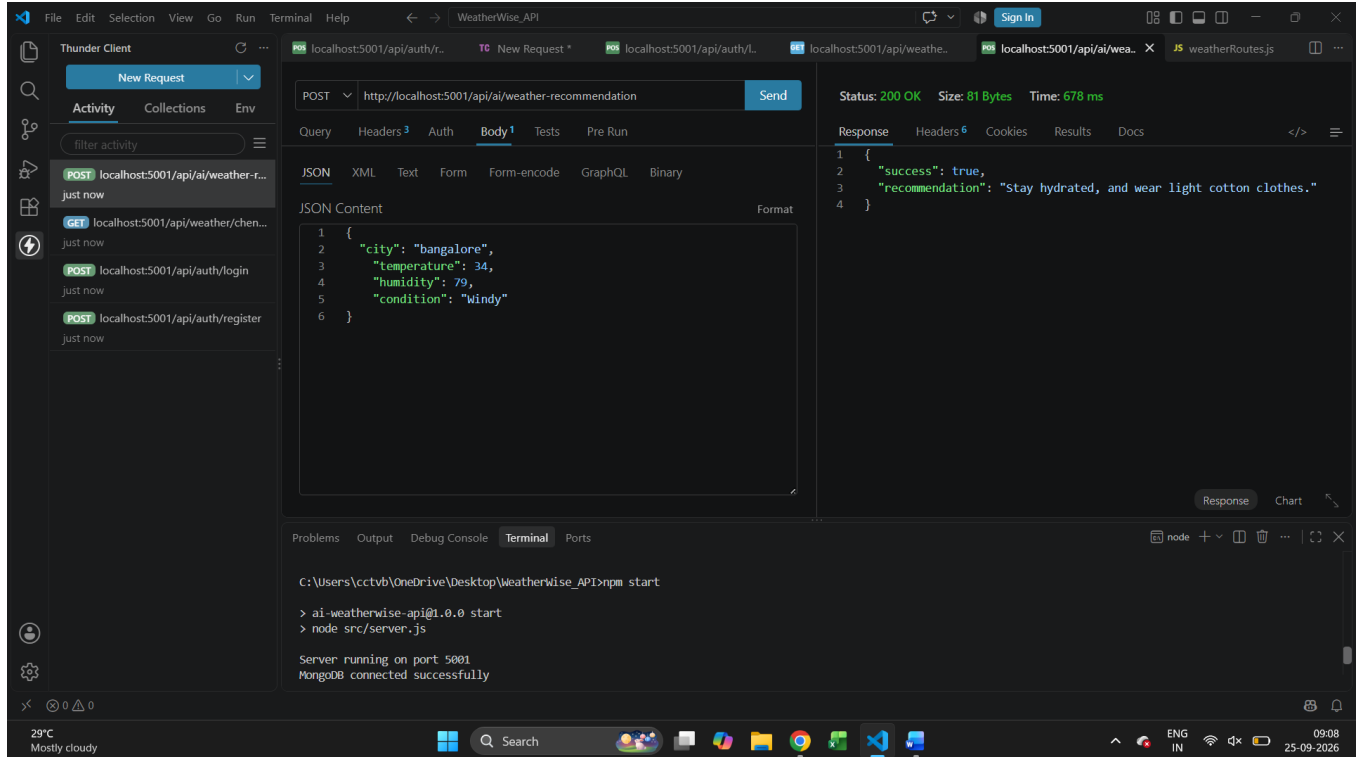
```
C:\Users\cctvb\OneDrive\Desktop\Weatherwise_API>npm start

> ai-weatherwise-api@1.0.0 start
> node src/server.js

Server running on port 5001
MongoDB connected successfully
```

The bottom of the image shows a Windows taskbar with the system clock at 09:08 on 25-09-2026 and weather information: 29°C, Mostly cloudy.

RECOMMENDATION API:



GitHub link:

https://github.com/Keerthanabca2024/Weatherwise_API.git

Demovideolink:

https://drive.google.com/file/d/15LLR7EEatBab71Hx9y04w0_bYSp-SdKZ/view?usp=drivesdk